

Laporan Tugas Besar 1
IF3270 PEMBELAJARAN MESIN
FEED-FORWARD NEURAL NETWORK
SEMESTER II TAHUN 2025/2026



Muhammad Izzat Jundy	13523092
Zulfaqqar Nayaka Athadiansyah	13523094
Muhammad Adha Ridwan	13523098

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

1	Deskripsi Persoalan	2
2	Analisis dan Pembahasan	2
2.1	Implementasi	2
2.2	Penjelasan Forward Propagation	3
3	Hasil Pengujian	3
3.1	CNN	3
3.2	Simple RNN dan LSTM	8
4	Penutup	8
4.1	Kesimpulan	8
4.2	Saran	8
4.3	Pembagian Tugas	8
	Referensi	9
A	Surat Pernyataan Penggunaan AI	10

1 Deskripsi Persoalan

2 Analisis dan Pembahasan

2.1 Implementasi

2.1.1 Modul CNN Layers

Berikut merupakan kelas-kelas layer yang diimplementasikan from scratch pada modul CNN.

Table 1: Atribut dan Metode Kelas-kelas CNN

Kelas	Atribut / Metode	Deskripsi
Conv2D	weight, bias stride, pad _forward(x) forward(x)	Parameter bobot konvolusi dan bias. Langkah dan padding operasi konvolusi. Operasi feedforward pada satu buah citra. Operasi konvolusi batch / channel support.
LocallyConnected2D	weight, bias, ... forward(x)	Bobot tidak dibagi ukurannya sesuai kernel/- patch. Forward propagation operasi locally con- nected.
Dense	weights, bias forward(x)	Bobot linear (fully connected layer). Feedforward ($X \cdot W + b$) dan aktivasi.
Flatten	forward(x) _forward(x)	Mengubah tensor multidimensi menjadi vek- tor 1D. Implementasi <code>reshape(-1)</code> .
MaxPooling2D	pool_size, stride forward(x)	Ukuran dan stride pooling filter. Melakukan max pooling window spatial.
AveragePooling2D	pool_size, stride forward(x)	Ukuran dan stride pooling filter. Melakukan average pooling window spatial.
GlobalAveragePooling2D	forward(x)	Mean pooling di seluruh dimensi spatial.
GlobalMaxPooling2D	forward(x)	Max pooling di seluruh dimensi spatial.

2.1.2 Modul RNN & LSTM Layers

Berikut merupakan kelas-kelas layer sekuensial yang diimplementasikan from scratch pada modul RNN dan LSTM.

Table 2: Atribut dan Metode Kelas-kelas RNN/LSTM

Kelas	Atribut / Metode	Deskripsi
SimpleRNNCell	<code>kernel</code>	Bobot rekursif dan input konvensional.
	<code>recurrent_kernel</code>	Bobot internal *hidden state* RNN.
SimpleRNN	<code>forward(x, t, h_prev)</code>	Perhitungan satu step state RNN.
	<code>cell</code>	Menggunakan SimpleRNNCell.
LSTMCell	<code>forward(x, init_st)</code>	Unroll / loop t sequence waktu untuk RNN.
	<code>kernel, bias</code>	Bobot gabungan (4 gates) dan bias.
LSTM	<code>recurrent_kernel</code>	Bobot berulang untuk state sebelumnya.
	<code>forward(x, h, c)</code>	Step kalkulasi input, forget, output, control gate. Menghasilkan sel dan *hidden* memori.
Embedding	<code>cell</code>	Menggunakan LSTMCell.
	<code>forward(x, init_st)</code>	Unroll step-by-step memory tracking.
Embedding	<code>weights</code>	Vector vocabulary size $\times$ dimension mapping.
	<code>forward(token_ids)</code>	Konversi integer token ids menjadi vektor dense.
	<code>from_keras(layer)</code>	Load transfer matriks embedding Keras.

2.2 Penjelasan Forward Propagation

2.2.1 CNN

2.2.2 RNN

2.2.3 LSTM

3 Hasil Pengujian

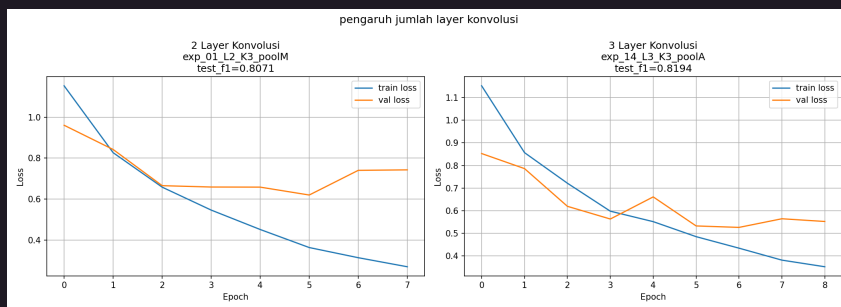
Bagian ini menyajikan hasil eksperimen model CNN serta RNN/LSTM yang dijalankan melalui Jupyter Notebook. Pengujian mencakup variasi arsitektur, parameter pelatihan, hingga perbandingan performa dengan pustaka standar Keras sebagai *benchmark*.

3.1 CNN

Berdasarkan hasil eksperimen pencarian parameter (*hyperparameter sweep*) 16 konfigurasi menggunakan pustaka Keras, diperoleh analisis dari berbagai komponen variasi arsitektur CNN sebagai berikut. Parameter performa utama yang diukur adalah F1-Score (*Macro*) pada himpunan data uji (*test set*) dan waktu pelatihan model.

3.1.1 Pengaruh Jumlah Layer Konvolusi

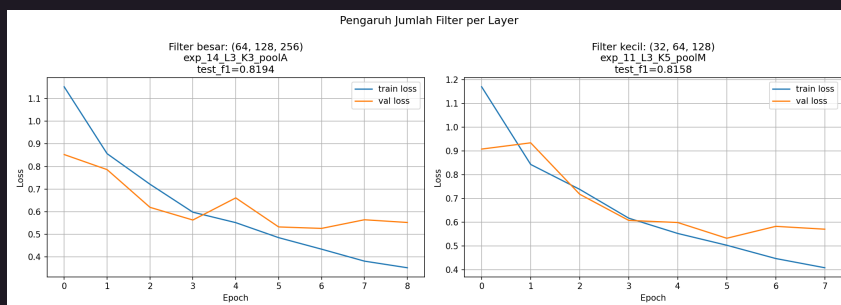
Peningkatan kedalaman arsitektur dari 2 layer menjadi 3 layer secara signifikan meningkatkan performa model (rata-rata F1-Macro naik dari 0.7701 menjadi 0.8009). Hal ini menunjukkan bahwa arsitektur yang lebih dalam mampu mengekstraksi representasi fitur hierarkis yang lebih kompleks. Namun, penambahan layer ini juga mengakibatkan peningkatan beban komputasi pelatihan, dari rata-rata 107.88 detik menjadi 153.75 detik.



Gambar 1: Rata-rata F1-Score berdasarkan Jumlah Layer

3.1.2 Pengaruh Kapasitas Model (Banyak Filter)

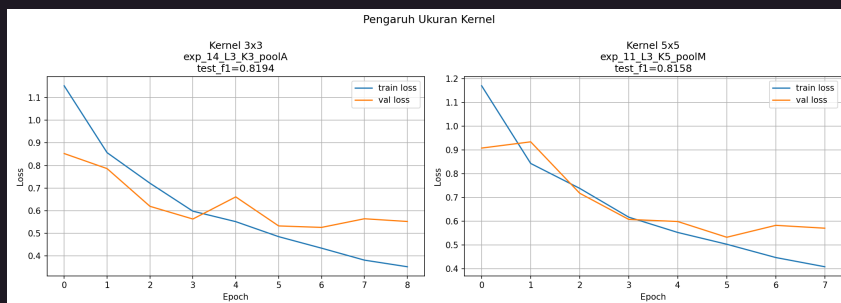
Arsitektur dengan kapasitas parameter filter kecil (dimulai dari 32 filter) secara mengejutkan sedikit mengungguli arsitektur filter besar (dimulai dari 64 filter) dengan perolehan skor F1-Macro 0.7895 berbanding 0.7815. Namun, model berkapasitas besar memakan waktu komputasi yang jauh lebih lama (rata-rata 173.38 detik) dibandingkan model yang lebih ringan (88.25 detik). Hal ini mengindikasikan adanya *diminishing return* atau sedikit *overfitting* pada model dengan filter yang terlalu banyak untuk dataset ini.



Gambar 2: Rata-rata F1-Score berdasarkan Kapasitas Filter

3.1.3 Pengaruh Ukuran Kernel

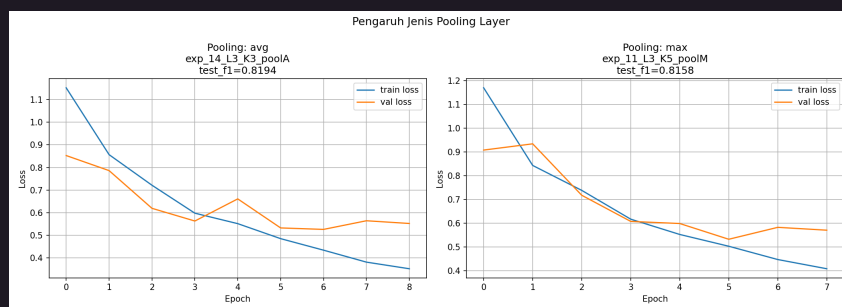
Eksperimen membuktikan bahwa penggunaan resolusi kernel konvolusi 3×3 memberikan performa yang lebih baik (F1-Macro 0.7900) daripada agregasi informasi yang dilakukan oleh ukuran 5×5 (F1-Macro 0.7810). Kernel 3×3 lebih unggul dalam menangkap detail spasial lokal. Model dengan kernel 3×3 juga lebih efisien dengan waktu komputasi latih rata-rata 115.50 detik berbanding 146.13 detik pada kernel 5×5 .



Gambar 3: Rata-rata F1-Score berdasarkan Ukuran Kernel

3.1.4 Pengaruh Jenis Pooling Layer

Model klasifikasi yang menerapkan *Max Pooling* secara rata-rata lebih superior (F1-Macro 0.7917) bila disandingkan dengan penerapan *Average Pooling* (0.7793). Hal ini dikarenakan operasi *max* mampu meretensi fitur yang paling menonjol (*salient features*) dari kumpulan aktivasi. *Max pooling* berjalan dengan rata-rata waktu latih 147.25 detik, sedangkan *average pooling* lebih cepat di 114.38 detik.



Gambar 4: Rata-rata F1-Score berdasarkan Jenis Pooling Layer

3.1.5 Perbandingan Keras vs *From Scratch*

Pengujian dilakukan untuk memvalidasi kebenaran implementasi sepadan (*equivalent*) antara pustaka Keras dan modul yang dibuat dari nol (*scratch*). Menggunakan konfigurasi terbaik (*exp_14*), diperoleh hasil F1-Macro yang hampir identik: 0.8194 (Keras) berbanding 0.8186 (*scratch*). Selisih yang sangat kecil (0.000879) ini membuktikan bahwa logika *forward propagation* yang diimplementasikan telah akurat. Namun, terdapat perbedaan performa waktu eksekusi yang drastis, di mana Keras hanya membutuhkan 1.8 detik untuk inferensi sedangkan modul *scratch* membutuhkan 154.1 detik akibat ketiadaan optimasi komputasi paralel tingkat rendah (GPU/XLA).

3.1.6 Perbandingan Arsitektur *Shared* vs *Non-shared*

Perbandingan dilakukan antara *Conv2D* (*shared weights*) dan *LocallyConnected2D* (*non-shared weights*). Hasil eksperimen menunjukkan bahwa arsitektur *shared weights* jauh lebih efisien dan memiliki performa lebih baik (F1-Macro 0.8186 dengan 10.9M parameter) dibandingkan arsitektur *non-shared* (F1-Macro 0.4255 dengan 18.9M parameter pada implementasi *scratch*). Hal ini memperkuat teori bahwa *translational invariance* yang dibawa oleh pembagian bobot sangat krusial untuk data gambar.



Gambar 5: Grafik Loss: Shared vs Non-shared (Locally Connected)

3.2 Simple RNN dan LSTM

3.2.1 Perbandingan RNN dan LSTM

3.2.2 Perbandingan dengan Keras

3.2.3 Pengaruh Panjang Maksimum Caption terhadap BLEU-4

4 Penutup

4.1 Kesimpulan

4.2 Saran

4.3 Pembagian Tugas

Table 3: Pembagian tugas anggota kelompok

NIM	Nama	Tugas
13523092	Muhammad Izzat Jundy	lorem ipsum
13523094	Zulfaqqar Nayaka Athadiansyah	lorem ipsum
13523098	Muhammad Adha Ridwan	lorem ipsum

Referensi

1. Karpathy, A. *The Spelled-Out Intro to Neural Networks and Backpropagation: Building Micrograd*. <https://youtu.be/VMj-3S1tku0>
2. Osajima, J. *Forward Propagation Explained*. <https://www.jasonosajima.com/forwardprop>
3. Osajima, J. *Backward Propagation Explained*. <https://www.jasonosajima.com/backprop>
4. NumPy Documentation. <https://numpy.org/doc/2.2/>
5. scikit-learn. *MLPClassifier*. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
6. OpenStax Calculus. *The Chain Rule for Multivariable Functions*. [https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/14%3ADifferentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariate_Functions](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3ADifferentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariate_Functions)
7. Green, E. *The Softmax Function and Its Derivative*. <https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>
8. Orr, D. *Building Autodiff from Scratch*. <https://douglasorr.github.io/2021-11-autodiff/article.html>
9. Grosse, R. *CSC2541 Tutorial: Automatic Differentiation*. https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf
10. Built In. *L2 Regularization*. <https://builtin.com/data-science/l2-regularization>

A Surat Pernyataan Penggunaan AI

Dengan ini, kami:

Nama/NIM : Zulfaqqar Nayaka Athadiansyah / 13523094
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 2: Convolutional Neural Network dan Recurrent Neural Network

menyatakan bahwa kami **menggunakan** AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika **Ya**, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: Grammarly, Paperpal, DeepL, Quillbot, ChatGPT, Gemini, dsb.	Tidak	
Pembuatan Teks: ChatGPT, Gemini, Copilot, Jasper, Jenni AI, dsb.	Tidak	
Bantuan Diskusi/Konten: Perplexity, ChatGPT, Zoom-Companion, dsb.	Ya	Untuk brainstorming awal, pembagian tugas, dan strategi implementasi
Grafik/Multimedia: DALL-E, Midjourney, Stable Diffusion, Canva AI, dsb.	Tidak	
Lainnya:	Tidak	

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini secara jujur dan transparan.

Bandung, 15 Mei 2026



Zulfaqqar Nayaka Athadiansyah